

Custom SQL Journey Documentation

Awwab Wadekar

June 22, 2026

1 Introduction

I was bored in my vacation, so I thought why not learn how SQL works under the hood with an attempt on a custom implementation?

This is torture. It was not fun.

Anyways the journey is a 5 phase mess:

Phase 1. The OS background

Phase 2. The Volcano Model

Phase 3. The System Catalog

Phase 4. The Parser Dilemma

Phase 5. The Editor Interface

2 Phase 1: The OS background

2.1 Day 1: Serialisation and Deserialisation

Data is often stored in Raw Bytes in Memory. So when we fetch it we actually can't use something like `struct.id` or `struct.name` to access fields.

Instead we use an offset variable to keep track of what we have accessed.

```
struct MyStruct{
    char name[30];
    int id;
}
```

Now here the total memory is $30 * 1 + 4 = 34$ bytes, as one char is 1 byte and one int is 4 bytes.

But usually the OS assigns Memory in terms of Pages, which here are considered to be 4 bytes in size. So you have Page 1...Page 2...Page 3.....Page 8...Page 9. Basically $9 * 4 = 36$ bytes.

Wait a minute, what ? I'm not a Mathematics Graduate but surely $36 > 34$.

So what actually happened here is padding, in the 8th Page, 2 bytes are used for the last 2 elements of the char array.

But we cant exactly put 2 bytes of an int in one page and 2 in another. And we for sure cannot have half a page. So the OS simply pads those 2 bytes up.

Well how do we fix this? The solution : Interpret everything as raw bytes and track offset of every field. Something along the lines of :

```
vector<uint8_t> buffer;
size_t offset = 0;
MyStruct obj;
obj.name = "Awwab";
obj.id = 77;
std::memcpy(buffer.data() + offset, &obj.name,
    sizeof(obj.name))
offset += sizeof(obj.name);
std::memcpy(buffer.data() + offset, &obj.id,
    sizeof(obj.id))
```

```
offset += sizeof(obj.id);
```

Similarly, Deserialization does the same process, just this time the source and destination are swapped.

```
vector<uint8_t> buffer;  
size_t offset = 0;  
MyStruct obj;  
std::memcpy(&obj.name, buffer.data() + offset,  
            sizeof(obj.name))  
offset += sizeof(obj.name);  
std::memcpy(&obj.id, buffer.data() + offset,  
            sizeof(obj.id))  
offset += sizeof(obj.id);
```